TEMPLE COLLEGE
WORKFORCE DEVELOPMENT

CHIEFOS

STUDENT REVIEW PACKET

# Week 13

Hosted vLLM endpoint: prerequisites, token, connect, models

Phase 3 — AI-Assisted Development · Weeks 13–18 · Modules 05–06 + Capstone

**No D2L login required.** This is the same lesson text posted in Brightspace for this week. Work the lessons in order on your own machine. Keep each checkpoint screenshot or pasted output. Bring it to the live session, or email it to your instructor, instead of submitting in D2L. Module quizzes stay in D2L and are not included here.

## In this packet


1. Module 05 — Continue.dev on the Shared vLLM Endpoint: overview
2. 05.00 Prerequisites Check
3. 05.01 The Inference Endpoint
4. 05.02 Connect to the Endpoint
5. 05.03 Explore the Models

# Module 05 — Continue.dev on the Shared vLLM Endpoint

### What you'll be able to do after this module

- Verify that the college's shared vLLM inference endpoint is reachable from your dev environment using a `curl` status check
- Query the endpoint's `/v1/models` route with the shared API key and identify the exact model ids it serves
- Send chat completions to Llama 3.3 70B over the OpenAI-compatible API and explain why Llama 3.1 8B is reserved for autocomplete
- Configure Continue.dev in VS Code via `~/.continue/config.yaml` with role-separated models pointing at the shared endpoint
- Chat about your FastAPI repo using the `@file`, `@codebase`, `@diff`, and `@terminal` context providers
- Use tab autocomplete to write a working route, reading every ghost-text suggestion before accepting it
- Edit highlighted code with `Ctrl+I` inline edits, accepting, rejecting, or partially accepting each proposed diff
- Use agent mode to make approval-gated, multi-file changes and ship a feature through a GitHub pull request

## Lessons and objectives

| # | Lesson | Learning objective |
|---|---|---|
| 00 | Prerequisites Check | Confirm the deliverables from Modules 00-04 (Git, Docker Compose, the FastAPI repo, and the GitHub remote) still work and that the Continue.dev extension is installed in VS Code. |
| 01 | The Inference Endpoint | Describe the college's shared vLLM server — its 2x RTX A6000 hardware, the two Llama models, and their roles — and confirm the endpoint answers with an HTTP status code using `curl`. |
| 02 | Connect to the Endpoint | Query the endpoint's `/v1/models` route with the shared API key via `curl` and confirm the exact Llama model ids appear in the OpenAI-style JSON response. |
| 03 | Explore the Models | Send real chat completions to the 70B model through the OpenAI-compatible `/v1/chat/completions` route, use a system prompt to steer its style, and explain why the 8B model is reserved for autocomplete. |
| 04 | Install + Configure Continue.dev | Install Continue.dev in VS Code and write `~/.continue/config.yaml` so the 70B serves chat, edit, and apply while the 8B serves autocomplete, then verify with a first chat and a ghost-text completion. |
| 05 | Chat | Use the `@file`, `@codebase`, `@diff`, and `@terminal` context providers to bring real repository context into Continue chat and get grounded answers about your FastAPI code. |
| 06 | Autocomplete | Write a working `PUT /items/{item_id}` route using tab autocomplete, reading each ghost-text suggestion before accepting with Tab, refining by typing, or dismissing with Esc. |
| 07 | Inline Edit | Use `Ctrl+I` inline edits to rename a function, add validation, extract a helper, and draft a test, reviewing every proposed diff before accepting, rejecting, or partially accepting it. |
| 08 | Agent Mode | Use Continue's agent mode to make approval-gated, multi-file changes on a feature branch, reviewing each tool call and rejecting steps that are wrong or destructive. |
| 09 | Final Checkpoint | Build a case-insensitive `GET /items/search` route with the Continue.dev + college vLLM stack, verify it with curl and Swagger UI, and ship it through a GitHub pull request. |

Each lesson is one learning object with a paired lab. Work in order — completing a lab unlocks the next lesson. The module ends with a **Check on Learning** quiz and a hands-on final checkpoint.

**Learning objective:** Confirm the deliverables from Modules 00-04 (Git, Docker Compose, the FastAPI repo, and the GitHub remote) still work and that the Continue.dev extension is installed in VS Code.

# Lesson 00: Prerequisites Check

**By the end, you will have confirmed every piece of Modules 00–04 still works, and that VS Code plus the Continue.dev extension are ready.**

~5 minutes

---

**1.** Open VS Code connected to your dev environment (WSL or SSH from Module 00). Open the integrated terminal.

**2.** Quick fire — confirm each Module's deliverable.

```
git --version
```

(Module 04 — should print 2.x.)

```
docker --version && docker compose version
```

(Module 02 — should print Docker + Compose versions.)

```
ls ~/docker-course/fastapi-app/main.py
```

(Module 03 — should print the path.)

```
git -C ~/docker-course/fastapi-app remote -v
```

(Module 04 — should print your GitHub origin URL.)

**3.** Confirm the FastAPI stack still starts.

```
cd ~/docker-course/fastapi-app
docker compose up -d
sleep 3
curl -s http://localhost:8000 | python3 -m json.tool
```

You should see `{"hello": "fastapi"}` (or whatever you renamed it to).

**4.** Stop it for now — we'll come back to it in lesson 05.

```
docker compose down
```

**5.** Confirm the Continue.dev extension is installed in VS Code. Open the extensions side bar (`Ctrl+Shift+X`), search for **Continue**, and confirm the extension published by **Continue** (id `continue.continue`) shows as installed. If it isn't, click **Install** now — lesson 04 configures it. (If you don't see it yet, don't worry; installing it here just saves a step later.)

**6.** Log it.

```
echo "Module 05 prereq check: passed $(date -Iseconds)" >> ~/docker-course/notes/notes.txt
```

---

## Watch out

- If any Module check fails, do not push forward. Wiring the editor to a remote AI endpoint is hard enough without other modules being broken. Fix the upstream module first.

---

## Checkpoint

```
git --version && docker --version && ls ~/docker-course/fastapi-app/main.py
```

Should print three things, no errors.

---

**Next up:** 01-the-inference-endpoint.md — Understand the shared inference endpoint

**Lab — do it for real.** Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

**Learning objective:** Describe the college's shared vLLM server — its 2x RTX A6000 hardware, the two Llama models, and their roles — and confirm the endpoint answers with an HTTP status code using `curl`.

# Lesson 01: The Inference Endpoint

**By the end, you will understand the shared inference endpoint the college runs — the hardware, the serving stack, the two models it hosts, and their roles — and you'll have confirmed your machine can reach it over the network.**

~15 minutes of reading + one connectivity check

> You are **not** running a model on your laptop. The college hosts **one shared inference server** on the campus network. Every student's editor points at the same box. Today we learn what that box is and confirm you can reach it. We install and configure nothing locally beyond the editor.

---

## The box

- **One shared server**, hosted by the college. You connect to it over the campus network (LAN or Tailscale). It is **not** a per-student local GPU, and it is not something you install.
- **Hardware:** 2× **NVIDIA RTX A6000** — 48 GB VRAM each, **~96 GB aggregate**.
- **Serving software: vLLM**, exposing an **OpenAI-compatible** HTTP API. The two A6000s run **tensor-parallel** (`--tensor-parallel-size 2`) so a single large model can be split across both cards.
- **Auth:** a **single shared API key** (`sk-college-shared`), or no key at all on the campus network. There are no per-student tokens to request — everyone uses the same endpoint.

### The endpoint URL

```
http://gpu.college.local:8000/v1
```

Some teams may instead see a Tailscale IP for the same box, e.g.:

```
http://100.68.103.54:8000/v1
```

Both point at the same server. Use whichever your instructor gives you. The `/v1` suffix is the OpenAI-compatible API path — the same shape the OpenAI SDK and Continue.dev expect.

---

## The two models

The endpoint serves **two** Llama models, each with a distinct job:

| Role in Continue | Model | vLLM model id | Status |
|---|---|---|---|
| **Autocomplete only** (tab / ghost-text) | **Llama 3.1 8B Instruct** | `meta-llama/Llama-3.1-8B-Instruct` | **live now** |
| **Chat, inline edit, agent** (everything else) | **Llama 3.3 70B Instruct** | `meta-llama/Llama-3.3-70B-Instruct` | **coming online** — tensor-parallel across both A6000s (quantized, e.g. AWQ/FP8, to fit ~96 GB) |

**Two models, two roles.** Big models (70B) are smart but slower — perfect for chat, edits, and agent work where you wait a couple of seconds for a good answer. Small models (8B) are fast — perfect for tab autocomplete where you need a suggestion in milliseconds. You'll wire both into Continue in lesson 04, selecting each by name.

> **Why Llama?** Both are Meta's **Llama** models: US-origin and permissively licensed, which satisfies the course policy of **no China-origin models**. (An earlier draft config used a China-origin model in violation of that policy — Llama fixes it.) You don't choose model sizes by your own VRAM anymore; the shared box already hosts the right ones.

---

## Confirm you can reach it

You don't need the model list yet (that's lesson 02) — just prove the box answers.

**1.** From your dev environment's integrated terminal, ping the API path and print only the HTTP status code:

```
curl -s -o /dev/null -w "%{http_code}\n" http://gpu.college.local:8000/v1/models
```

(If your instructor gave you the Tailscale IP instead, substitute it for `gpu.college.local`.)

**2.** Read the result:

- `200` — the box answered and no key is required.
- `401` — the box answered but wants the shared key. **This still means success for this lesson** — you reached the server. You'll supply the key in lesson 02.
- `000` **or a hang** — you did **not** reach the server. You're likely off the campus network. Connect to the LAN / start Tailscale, then retry. If it still fails, ask the instructor — they own the box.

**3.** Log it.

```
echo "Inference endpoint reachable: http://gpu.college.local:8000/v1 ($(date -Iseconds))" >> ~/docker-course/notes/notes.txt
```

---

## Watch out

- **Nothing to install here.** No CUDA, no drivers, no `nvidia-smi`, no local GPU. If you find yourself installing an inference engine on your laptop, stop — you've misread the module. The server is already running.
- **It's a network dependency now.** The endpoint has to be reachable. Off-campus? You need Tailscale (or the VPN your instructor specifies). A `000` status is a network problem, not a model problem.
- **Plain HTTP.** The URL is `http://…`, not `https://` — it's an internal campus service. That's expected.
- **The 70B may be "coming online."** If chat/edit later report the 70B isn't available yet, the 8B autocomplete model is still live. Confirm current status with your instructor.
- **Shared box, shared load.** Everyone in the cohort hits the same server. If responses feel slow during a busy lab, that's contention, not a broken setup.

---

## Checkpoint

```
curl -s -o /dev/null -w "%{http_code}\n" http://gpu.college.local:8000/v1/models
```

Should print an HTTP status code (`200` or `401` — both mean the server answered). A `000` or a timeout means you can't reach the box; fix your network before lesson 02.

You should also be able to state, without looking: which model does **autocomplete** (Llama 3.1 8B) and which does **chat/edit/agent** (Llama 3.3 70B).

---

**Next up:** 02-connect-to-endpoint.md — curl the shared endpoint and confirm the model list

**Lab — do it for real.** Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

**Learning objective:** Query the endpoint's `/v1/models` route with the shared API key via `curl` and confirm the exact Llama model ids appear in the OpenAI-style JSON response.

# Lesson 02: Connect to the Endpoint

**By the end, you will have queried the shared vLLM endpoint's `/v1/models` route with the shared key and confirmed both Llama models appear in the list.**

~15 minutes of typing

> The endpoint speaks the **OpenAI-compatible** API. Before we wire up the editor, we prove the connection by hand with `curl` — list the models the server is serving, using the shared base URL and key. If `curl` can see the models, Continue.dev will too.

---

**1.** Make sure your venv from Module 03 is **not** active (clearer terminal output).

```
deactivate 2>/dev/null
```

**2.** Set the endpoint and shared key as shell variables so the rest of the lesson is copy-paste. (Substitute the Tailscale IP for `gpu.college.local` if that's what your instructor gave you.)

```
export VLLM_BASE=http://gpu.college.local:8000/v1
export VLLM_KEY=sk-college-shared
```

> If your instructor said **no key is required** on the campus network, you can leave `VLLM_KEY` set to anything — the server will ignore the `Authorization` header. Setting it does no harm.

**3.** List the models the endpoint is serving.

```
curl -s "$VLLM_BASE/models" -H "Authorization: Bearer $VLLM_KEY" | python3 -m
json.tool
```

**4.** You should see an OpenAI-style response — an `object: "list"` with a `data` array of model records:

```
{
    "object": "list",
    "data": [
        {
            "id": "meta-llama/Llama-3.1-8B-Instruct",
            "object": "model",
            "owned_by": "vllm"
        },
        {
            "id": "meta-llama/Llama-3.3-70B-Instruct",
            "object": "model",
            "owned_by": "vllm"
        }
    ]
}
```


**5.** Read the `id` fields — these are the **exact** model ids you'll put in your Continue config in lesson 04:

- `meta-llama/Llama-3.1-8B-Instruct` — the **autocomplete** model (live now).
- `meta-llama/Llama-3.3-70B-Instruct` — the **chat / edit / agent** model.

> The 70B is **coming online** (tensor-parallel across both A6000s). If it isn't in the list yet, you'll still see the 8B — that's enough to continue. Ask the instructor whether the 70B is up before lesson 05.

**6.** Pull just the ids out, to be sure (no eyeballing the whole blob):

```
curl -s "$VLLM_BASE/models" -H "Authorization: Bearer $VLLM_KEY" \
  | python3 -c "import sys,json; [print(m['id']) for m in json.load(sys.stdin)
['data']]"
```

```
meta-llama/Llama-3.1-8B-Instruct
meta-llama/Llama-3.3-70B-Instruct
```

**7.** Log it.

```
echo "vLLM endpoint connected: $VLLM_BASE (models: $(curl -s "$VLLM_BASE/models" -H
"Authorization: Bearer $VLLM_KEY" | python3 -c "import sys,json; print(',
'.join(m['id'] for m in json.load(sys.stdin)['data']))"))" >> ~/docker-
course/notes/notes.txt
```

---

## Watch out

- **`000` / connection refused / timeout** — you can't reach the box. This is a **network** problem, not a key problem. Confirm you're on the campus LAN or Tailscale (lesson 01), then retry.
- **`401 Unauthorized`** — the server wants the shared key and yours is wrong or missing. Check the `Authorization: Bearer $VLLM_KEY` header and confirm `VLLM_KEY` matches the value your instructor gave you (`sk-college-shared` by default).
- **`404` on `/models`** — you probably dropped the `/v1` from the base URL. The path is `http://gpu.college.local:8000/v1/models`.
- **Empty `data` array** — the server is up but no model is loaded yet (the 70B may still be warming up). The 8B should always be present; if even it is missing, ask the instructor.
- **Trailing slashes.** `$VLLM_BASE` should end in `/v1` with **no** trailing slash, so `$VLLM_BASE/models` becomes `.../v1/models`.

---

## Handy commands

| Task | Command |
|---|---|
| List models | `curl -s "$VLLM_BASE/models" -H "Authorization: Bearer $VLLM_KEY"` |
| Just the ids | pipe the above through the `json` one-liner in step 6 |
| Check reachability only | `curl -s -o /dev/null -w "%{http_code}\n" "$VLLM_BASE/models"` |

---

## Checkpoint

```
curl -s "$VLLM_BASE/models" -H "Authorization: Bearer $VLLM_KEY" | grep -q "Llama-3.1-8B-Instruct" && echo "OK: endpoint serving models"
```

Should print `OK: endpoint serving models`.

In lesson 03 you will send an actual chat completion to the 70B model and see it generate — this lesson just proves the connection and lists what's available.

---

**Next up:** 03-explore-the-models.md — Send a test chat completion to the 70B model

**Lab — do it for real.** Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

**Learning objective:** Send real chat completions to the 70B model through the OpenAI-compatible `/v1/chat/completions` route, use a system prompt to steer its style, and explain why the 8B model is reserved for autocomplete.

# Lesson 03: Explore the Models

**By the end, you will have sent a real chat completion to the 70B model over the OpenAI-compatible API, felt its latency, and confirmed the 8B model is autocomplete-only.**

~20 minutes of typing + reading

> **Two models, two roles.** The **70B** (Llama 3.3 70B Instruct) is the smart, conversational model — it handles chat, inline edit, and agent mode. The **8B** (Llama 3.1 8B Instruct) is fast and lightweight — it's strictly the tab-autocomplete engine. Today we exercise the 70B directly with `curl`, the same API Continue.dev will use in lesson 04.

**All calls reuse the shell variables from lesson 02.** If you opened a fresh terminal, set them again:

```
export VLLM_BASE=http://gpu.college.local:8000/v1
export VLLM_KEY=sk-college-shared
```

---

## The two roles, restated

| Model | vLLM model id | Role | Why |
|---|---|---|---|
| Llama 3.3 70B Instruct | `meta-llama/Llama-3.3-70B-Instruct` | chat, edit, agent | Large, reasoning-capable; worth the extra second. |
| Llama 3.1 8B Instruct | `meta-llama/Llama-3.1-8B-Instruct` | autocomplete only | Small and fast; returns a suggestion in milliseconds. |

You do **not** pick a model by your own hardware — the shared box already hosts both. You just address each by its `id`.

---

**1.** Send a first chat completion to the **70B**. This is the OpenAI-compatible `/v1/chat/completions` route.

```
curl -s "$VLLM_BASE/chat/completions" \
  -H "Authorization: Bearer $VLLM_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "meta-llama/Llama-3.3-70B-Instruct",
    "messages": [
      {"role": "user", "content": "What does the FastAPI Depends() function do, in
one sentence?"}
    ]
  }' | python3 -c "import sys,json; print(json.load(sys.stdin)['choices'][0]
['message']['content'])"
```

You should see a multi-second pause (the 70B is large and shared), then a coherent one-sentence answer printed. **That's the model Continue will use for chat.**

**2.** Feel the shape of the request. Notice three things:

- `"model"` names the exact id from lesson 02.
- `"messages"` is a list of role/content turns — same format the OpenAI SDK uses.
- The answer comes back at `choices[0].message.content`.

**3.** Try a system prompt to steer the style — models take a `system` turn before the `user` turn.

```
curl -s "$VLLM_BASE/chat/completions" \
  -H "Authorization: Bearer $VLLM_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "meta-llama/Llama-3.3-70B-Instruct",
    "messages": [
      {"role": "system", "content": "You are a terse senior Python engineer. Answer
in one line of code, no prose."},
      {"role": "user", "content": "A FastAPI route that returns {\"ok\": true}."}
    ]
  }' | python3 -c "import sys,json; print(json.load(sys.stdin)['choices'][0]
['message']['content'])"
```

You should get essentially a single line of Python. Same model, very different behavior — that's the system prompt at work.

**4.** Ask a follow-up-style question in one call to feel the model's coding ability:

```
curl -s "$VLLM_BASE/chat/completions" \
  -H "Authorization: Bearer $VLLM_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "meta-llama/Llama-3.3-70B-Instruct",
    "messages": [
      {"role": "user", "content": "Show me a tiny example of FastAPI Depends() with
a dependency function."}
    ]
  }' | python3 -c "import sys,json; print(json.load(sys.stdin)['choices'][0]
['message']['content'])"
```

Read the code it returns. This is the quality you'll get in the chat panel in lesson 05.

**5.** Now look at the **8B**. It *can* answer a chat prompt, but in this course it is reserved for autocomplete. Prove it responds, then leave it alone:

```
curl -s "$VLLM_BASE/chat/completions" \
  -H "Authorization: Bearer $VLLM_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "meta-llama/Llama-3.1-8B-Instruct",
    "messages": [
      {"role": "user", "content": "Complete this function:\ndef fib(n):"}
    ]
  }' | python3 -c "import sys,json; print(json.load(sys.stdin)['choices'][0]
['message']['content'])"
```

It answers, and faster than the 70B — but notice it's a smaller, less careful model. In Continue we give it **only** the `autocomplete` role, where speed matters and the context is tightly scoped. Chat/edit/agent all go to the 70B.

> **Why 8B for autocomplete and 70B for everything else?** Tab autocomplete fires on every pause in your typing — it must return in milliseconds, so it needs the small, fast model. Chat, inline edit, and agent mode run once when you ask, and reward reasoning — so they get the big model even though it's a beat slower. Same reason you'd never wait two seconds for every ghost-text suggestion.

**6.** Log it.

```
echo "Models explored: chat-completions OK against Llama-3.3-70B (chat/edit/agent),
Llama-3.1-8B reserved for autocomplete" >> ~/docker-course/notes/notes.txt
```

## Watch out

- **First response is slow.** The 70B is large and shared across the whole cohort. Expect a few seconds before text comes back, longer during a busy lab. Don't conclude it's broken until you've waited.
- **`model not found` / `404` on the 70B.** The 70B is **coming online** — mid-migration it may not be loaded yet. Confirm with your instructor. In the meantime the 8B is live, so the API itself works.
- **`401 Unauthorized`.** The shared key is wrong or missing — recheck `VLLM_KEY` and the `Authorization` header (lesson 02).
- **Don't put the 8B on chat in Continue.** It's the autocomplete model. Using it for chat gives weaker answers; that's why lesson 04's `roles:` keep chat/edit/agent on the 70B.
- **It's shared, not private.** Everything you send goes to the college server. Don't paste secrets — build the hygiene habit now (it carries into lesson 05).

---

## Checkpoint

```
curl -s "$VLLM_BASE/chat/completions" \
  -H "Authorization: Bearer $VLLM_KEY" -H "Content-Type: application/json" \
  -d '{"model":"meta-llama/Llama-3.3-70B-Instruct","messages":
[{"role":"user","content":"What is 2+2? One word."}]}' \
  | python3 -c "import sys,json; print(json.load(sys.stdin)['choices'][0]
['message']['content'])"
```

Should print `4` (or `Four`). If the 70B isn't online yet, run the same call with `meta-llama/Llama-3.1-8B-Instruct` to confirm the API works, and check 70B status with the instructor.

You should also be able to say why the 8B is the autocomplete model and the 70B does everything else.

---

**Next up:** 04-install-and-configure-continue.md — Wire Continue.dev to the vLLM endpoint

**Lab — do it for real.** Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.